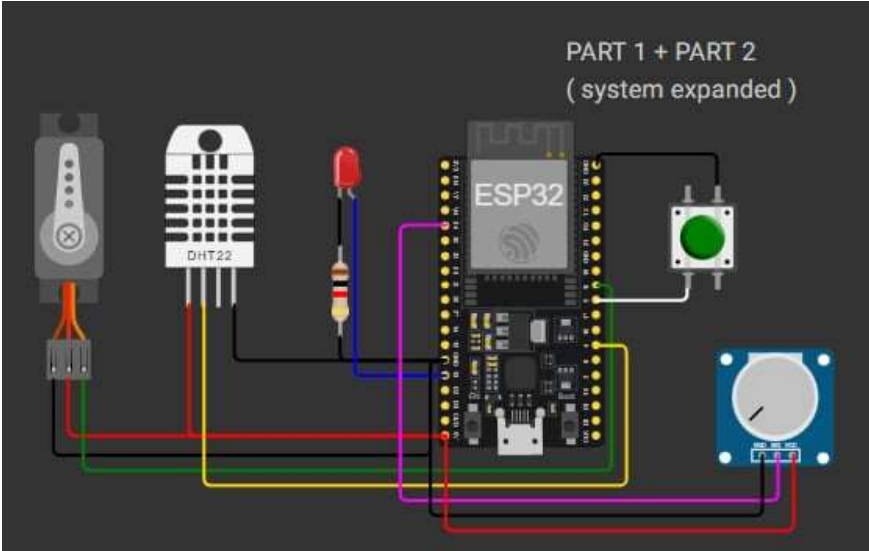
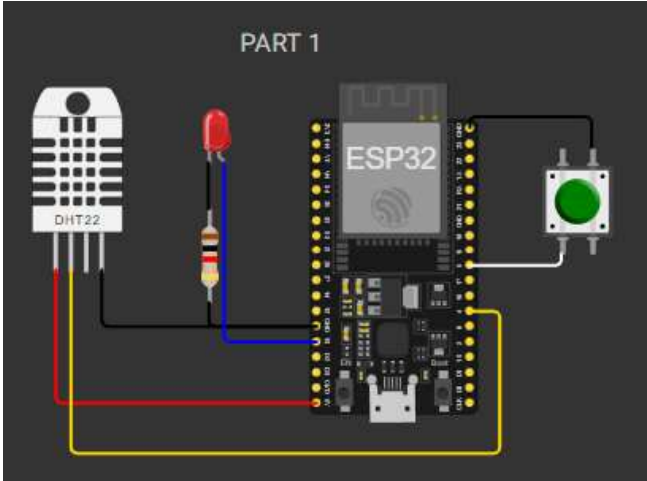


Embedded Systems Midterm Exam Independent Practical Assessment BSCPE

Circuit Documentation



PART 1	
Component	Specification / Description
ESP32 Dev Board	Dual-core MCU, WiFi/Bluetooth, multiple GPIOs, ADC, PWM, I2C, UART
DHT22 Sensor	Digital temperature & humidity sensor, $\pm 0.5^{\circ}\text{C}$ accuracy, 3–5V
LED	Indicator LED, typically 5mm,
Resistor	Current-limiting resistor for LED
Push button	Tactile push-button with debounce handling

PART 1	
Component	Pin Terminal for ESP32
DHT22 Sensor	GPIO 4
LED	GPIO 13
Resistor	Connected to GND of LED
Push button	GPIO 5

PART 1 + PART 2 (SYSTEM EXPANDED)	
Component	Specification / Description
ESP32 Dev Board	Dual-core MCU, WiFi/Bluetooth, multiple GPIOs, ADC, PWM, I2C, UART
DHT22 Sensor	Digital temperature & humidity sensor, ±0.5°C accuracy, 3–5V
LED	Indicator LED, typically 5mm,
Resistor	Current-limiting resistor for LED
Push button	Tactile push-button with debounce handling
Servo Motor (SG90)	180° rotation, PWM control, 5V power
Potentiometer	Linear rotary variable resistor for analog input

PART 1 + PART 2 (SYSTEM EXPANDED)	
Component	Pin Terminal for ESP32
DHT22 Sensor	GPIO 4
LED	GPIO 13
Resistor	Connected to GND of LED
Push button	GPIO 5
Servo Motor (SG90)	GPIO 18
Potentiometer	GPIO 34

Written Analysis

Evaluation of System Performance

The system was successfully implemented in two phases: data acquisition and actuator control. In Part 1, the DHT22 sensor effectively measured ambient temperature and humidity. The moving average filter (10-point buffer) helped eliminate noise and sudden fluctuations in readings, resulting in stable and reliable data output. The heat index calculation added context to the temperature and humidity data, especially helpful in understanding perceived environmental conditions. Visual indication using an LED based on temperature thresholds functioned accurately. The hysteresis mechanism prevented LED flickering when the average temperature hovered near the threshold, showcasing good state stability.

In **Part 2**, the servo motor was controlled using a potentiometer; however, the response between the potentiometer adjustment and servo movement showed noticeable delay. While the system successfully mapped analog input to servo angles, the presence of sensor reading intervals and timing constraints introduced lag in responsiveness. This made the interaction feel less immediate and highlighted a need for separating sensor and actuator tasks more efficiently. The system still used a physical button to toggle between modes (0: Temp Only, 1: Full Sensor Info, 2: Servo Control), with software-based debouncing ensuring stable mode switching.

Limitations and Potential Improvements

1. **Sensor Reading Delay:** The DHT22 sensor has a relatively slow response time (around 2 seconds), which limits how frequently data can be refreshed. Using a faster sensor like a BME280 or DS18B20 could improve responsiveness and allow for more frequent updates.
2. **Limited Interface:** The serial monitor is the only interface to display sensor values. While useful for testing, adding an OLED or LCD display would make the device standalone and more user-friendly.
3. **Servo Power Considerations:** The servo motor requires more current than the ESP32's onboard 3.3V regulator can safely provide. Powering the servo directly from the ESP32 may cause voltage drops, leading to unexpected resets or erratic behavior. To ensure stable operation, the servo should be powered using an external **5V power source**, with a shared ground connection to the ESP32.
4. **No Data Logging:** The current system displays real-time data but does not log or store historical values. Incorporating an SD card module or using Wi-Fi to send data to a server (e.g., Blynk) would enhance functionality.
5. **Basic Control Logic:** The control system is simple and lacks feedback-based automation (e.g., closing a vent if temperature exceeds a setpoint). Implementing proportional or PID control could bring advanced system behavior.

Reflection and Learning Outcomes

Through this project, I learned how to design and implement a multi-functional embedded system using the ESP32. It involved:

- Integrating multiple types of inputs (digital, analog, and sensor-based)
- Handling hardware interrupts and debouncing in real-time applications
- Applying data filtering techniques like moving averages
- Creating multi-mode functionality based on user input
- Managing servo control via analog input in a feedback-friendly environment

I developed a better understanding of code optimization particularly how delays impact performance in real-time systems. By removing unnecessary delays, servo responsiveness was greatly improved.

The project also strengthened my debugging skills. Managing different modules (DHT22, servo, potentiometer, button) and ensuring they worked together smoothly helped me learn how to break down and isolate problems in embedded systems.

I appreciated how software structure (modes, state management, filtering) can make hardware more intelligent and adaptable. Overall, the project gave me a solid foundation in IoT system design, sensor integration, actuator control, and real-time programming.